Assignment 1: Data Ingestion and Storage

1. Aim

To understand and implement **data ingestion and storage in a Big Data environment** by:

- Setting up Hadoop or using a cloud-based Big Data service.
- Uploading a large CSV/JSON dataset into **HDFS**.
- Converting the dataset into **Parquet and ORC** formats.
- Comparing storage size and query performance.
- Performing data transformations using **Apache Spark**.
- Storing the processed data in **MongoDB**, a NoSQL database.

2. Learning Objectives

After completing this practical, students should be able to:

1. Understand the concept of Big Data ingestion.
2. Understand HDFS and its architecture.
3. Upload files into HDFS.
4. Read CSV/JSON data using Spark.
5. Convert data into Parquet and ORC formats.
6. Compare different storage formats.
7. Perform basic transformations using Spark.
8. Write processed data into MongoDB.
9. Understand the role of NoSQL databases in Big Data applications.

3. Software Requirements

Option A — Recommended for beginners

Use **Docker + Hadoop + Spark + MongoDB**.

Option B

Use a cloud-based platform such as:

- Google Cloud Dataproc
- AWS EMR
- Azure HDInsight

Option C

Install Hadoop and Spark directly on a computer.

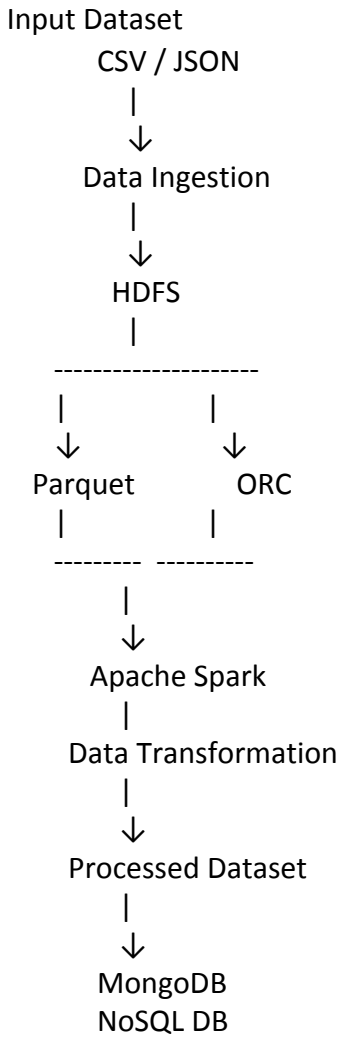
For a college practical, **Option A or a managed cloud service is generally easier than manually configuring a multi-node Hadoop cluster.**

4. Technologies Used

Technology	Purpose
Hadoop	Big Data framework
HDFS	Distributed file storage
Apache Spark	Data processing
MapReduce	Distributed processing
CSV/JSON	Input data
Parquet	Columnar storage
ORC	Columnar storage
MongoDB	NoSQL database
Python	Programming language
PySpark	Python API for Spark

5. Basic Architecture

The overall practical can be understood as:



6. Understanding HDFS

HDFS stands for **Hadoop Distributed File System**.

It is the storage component of Hadoop designed to store very large datasets across multiple machines.

HDFS mainly contains:

1. NameNode

The NameNode manages:

- File names
- Directory structure
- File metadata
- Locations of data blocks

2. DataNode

DataNodes actually store the data blocks.

Example:

Suppose we have a large file:

sales.csv

Instead of storing the entire file on one machine, HDFS can divide it into blocks:

```
sales.csv
        |
        +---- Block 1
        +---- Block 2
        +---- Block 3
        +---- Block 4
```

These blocks can be distributed among DataNodes.

7. Dataset Selection

For this practical, you can use a CSV dataset such as:

`sales.csv`

Example:

transaction_id	product	category	quantity	price	city
1001	Laptop	Electronics	2	55000	Pune
1002	Mouse	Electronics	5	800	Mumbai
1003	Chair	Furniture	3	4500	Delhi
1004	Keyboard	Electronics	4	1500	Pune

For a real Big Data experiment, use a considerably larger dataset.

Important: A small dataset can be used for learning the commands, but it will not provide a meaningful comparison of distributed-storage performance.

PART A — SETTING UP HADOOP

8. Verify Hadoop Installation

Open Command Prompt/Terminal and execute:

</Bash>
```
hadoop version
```

Expected output will contain something similar to:

```
Hadoop 3.x.x
```

Also verify Java:
</Bash>
```
java -version
```

9. Start Hadoop Services

For a pseudo-distributed Hadoop installation, start HDFS using:

</Bash>
```
start-dfs.sh
```

If using Windows-specific Hadoop configuration, the commands may differ depending on the Hadoop distribution.

Check running processes:

</Bash>
```
Jps
```

You should see services such as:

```
NameNode
DataNode
SecondaryNameNode
```

10. Create a Directory in HDFS
First create a directory for the practical:

</Bash>
```
hdfs dfs -mkdir -p /bigdata/assignment1/input
```

Check the directory:
</Bash>
```
hdfs dfs -ls /bigdata/assignment1
```

Expected:
```
input
```

11. Upload CSV Dataset to HDFS

Suppose the dataset is stored locally as:

```
sales.csv
```

Upload it:

</Bash>

```
hdfs dfs -put sales.csv /bigdata/assignment1/input/
```

Verify:

</Bash>

```
hdfs dfs -ls /bigdata/assignment1/input
```

Expected:

```
sales.csv
```

12. View the Dataset in HDFS

Use:

</Bash>

```
hdfs dfs -cat /bigdata/assignment1/input/sales.csv
```

For a large file, it is better to use:

</Bash>

```
hdfs dfs -head /bigdata/assignment1/input/sales.csv
```

This displays the beginning of the file.

13. Important HDFS Commands

Students should learn these commands:

Command	Purpose
`hdfs dfs -ls`	List files
`hdfs dfs -mkdir`	Create directory
`hdfs dfs -put`	Upload file
`hdfs dfs -get`	Download file
`hdfs dfs -cat`	Display file
`hdfs dfs -rm`	Delete file
`hdfs dfs -du`	Check disk usage
`hdfs dfs -cp`	Copy file
`hdfs dfs -mv`	Move file

Example:

</Bash>

```
hdfs dfs -du -h /bigdata/assignment1/input
```

PART B — DATA PROCESSING USING SPARK

14. What is Apache Spark?

Apache Spark is a distributed data-processing framework.

It can process large datasets much faster than traditional disk-based MapReduce for many workloads because Spark can perform computations efficiently in memory.

For this practical, we will use **PySpark**.

15. Install PySpark
If PySpark is not installed:
</Bash>

```
pip install pyspark
```

Verify:
</Bash>

```
python -c "import pyspark; print(pyspark.__version__)"
```

6. Create a Spark Program

Create a file:

`assignment1.py`

Basic Spark program:

```
</Python>

from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Assignment1") \
    .getOrCreate()

print("Spark started successfully")

spark.stop()
```

Run:

</Bash>

```
python assignment1.py
```

After Run

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Assignment1") \
    .getOrCreate()

print("Spark started successfully")

spark.stop()
```

17. Read CSV from HDFS

Use the following PySpark program:

<Python>

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("DataIngestion") \
    .getOrCreate()

df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv("hdfs://localhost:9000/bigdata/assignment1/input/sales.csv")

df.show()

df.printSchema()

spark.stop()
```

#After Run

Run started
Initializing environment
Installing packages
Running code
ModuleNotFoundError: No module named 'pyspark'
Run completed in 1422.7000000476837ms

Explanation

<Python>

SparkSession.builder

#After Run
Run started
Initializing environment
Installing packages
Running code
NameError: name 'SparkSession' is not defined
Run completed in 1013ms

Creates a Spark session.

<Python>

.option("header", True)

#After Run

Run started
Initializing environment
Installing packages
Running code
SyntaxError: invalid syntax (<exec>, line 1)
Run completed in 1086ms

Tells Spark that the first row contains column names.

<Python>

```
.option("inferSchema", True)
```

#After Run
Run started
Initializing environment
Installing packages
Running code
SyntaxError: invalid syntax (<exec>, line 1)
Run completed in 1050.300000011921ms

Automatically detects data types.

<Python>

```
.csv(...)
```

Reads the CSV file.

<Python>

```
df.show()
```

Displays records.

<Python>

```
df.printSchema()
```

Displays column names and data types.

PART C — PARQUET FORMAT

18. What is Parquet?

Apache Parquet is a columnar storage format designed for efficient Big Data processing.

Instead of storing data row by row:

```
Row 1
Row 2
Row 3
Row 4
```

Parquet stores data in a column-oriented structure.

For example:

```
transaction_id column
product column
quantity column
price column
city column
```

This makes it efficient when queries require only a few columns.

19. Convert CSV to Parquet

Add:

<Python>

```
df.write \
    .mode("overwrite") \

.parquet("hdfs://localhost:9000/bigdata/ass
ignment1/parquet")
```

#After Run

Complete example:
<Python>

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("CSVToParquet") \
    .getOrCreate()

input_path = "hdfs://localhost:9000/bigdata/assignment1/input/sales.csv"
parquet_path = "hdfs://localhost:9000/bigdata/assignment1/parquet"

df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv(input_path)

df.write \
    .mode("overwrite") \
    .parquet(parquet_path)

print("CSV successfully converted to Parquet")

spark.stop()
```

#After Run

20. Read Parquet Data

<Python>

```
df_parquet = spark.read.parquet(

"hdfs://localhost:9000/bigdata/assignment1/
parquet"
)

df_parquet.show()
```

PART D — ORC FORMAT

21. What is ORC?

ORC stands for **Optimized Row Columnar**.

It is another columnar storage format commonly used with Hadoop-based data-processing systems.

Advantages include:

- Compression
- Efficient storage
- Fast analytical queries
- Predicate pushdown
- Column pruning

22. Convert CSV to ORC

<Python>

```
df.write \
    .mode("overwrite") \

.orc("hdfs://localhost:9000/bigdata/assignment1/orc")
```

#After Run

Complete code:

<Python>

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("CSVToORC") \
    .getOrCreate()

input_path =
"hdfs://localhost:9000/bigdata/assignment1/input/sales.csv"

df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv(input_path)

df.write \
    .mode("overwrite") \
    .orc("hdfs://localhost:9000/bigdata/assignment1/orc")

print("CSV successfully converted to ORC")

spark.stop()
```

#After Run

PART E — COMPARE STORAGE SIZE

23. Check CSV Size

Run:

<Bash>

```
hdfs dfs -du -h /bigdata/assignment1/input
```

Record the size.

Example:

```
CSV = 500 MB
```

24. Check Parquet Size

<Bash>

```
hdfs dfs -du -h
/bigdata/assignment1/parquet
```

Example:

```
Parquet = 120 MB
```

25. Check ORC Size

<Bash>

```
hdfs dfs -du -h /bigdata/assignment1/orc
```

Example:

```
ORC = 100 MB
```

These numbers are only examples. Students must record the actual values from their system.

26. Prepare Comparison Table

Format	Storage Size	Compression	Query Performance
CSV	500 MB	Low/None	Slow
Parquet	120 MB	High	Fast
ORC	100 MB	High	Fast
JSON	650 MB	Low/None	Relatively slow

Students can prepare a table like this:
Again, these values are illustrative.

PART F — QUERY PERFORMANCE

27. Query CSV

For example:

<Python>

```
df.filter(df.city == "Pune").count()
```

#After Run

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 1007.1000000238419ms

Measure execution time:

<Python>

```
import time

start = time.time()

count = df.filter(df.city ==
"Pune").count()

end = time.time()

print("Count:", count)
print("Time:", end - start)
```

#After Run
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df' is not defined
Run completed in 1506.8999999761581ms

28. Query Parquet

<Python>

```
import time

start = time.time()

count = df_parquet.filter(
    df_parquet.city == "Pune"
).count()

end = time.time()

print("Count:", count)
print("Time:", end - start)
```

#After Run

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df_parquet' is not defined

Run completed in 1441.300000011921ms

29. Query ORC

<Python>

```
df_orc = spark.read.orc(

"hdfs://localhost:9000/bigdata/assignment1/
orc"
)

start = time.time()

count = df_orc.filter(
    df_orc.city == "Pune"
).count()

end = time.time()

print("Count:", count)
print("Time:", end - start)
```

#After Run
Run started
Initializing environment
Installing packages
Running code
NameError: name 'spark' is not defined
Run completed in 1013.1999999880791ms

30. Expected Observation

Usually, for analytical queries over sufficiently large datasets:

```
CSV
 ↓
More data read
 ↓
More processing
 ↓
Slower

Parquet / ORC
 ↓
Column pruning + compression
 ↓
Less data read
 ↓
Faster
```

However, **the exact performance depends on dataset size, hardware, Spark configuration, caching, compression, partitioning, and query design.**

PART G — DATA TRANSFORMATION

Now we perform basic transformations using Spark.

Suppose our dataset contains:

```
transaction_id
product
category
quantity
price
city
```

31. Calculate Total Amount
Add a new column:

<Python>

```
from pyspark.sql.functions import col

df = df.withColumn(
    "total_amount",
    col("quantity") * col("price")
)
```

#After Run
Run started
Initializing environment
Installing packages

Display:
<Python>

```
df.show()
```

#After Run
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df' is not defined
Run completed in 1100.800000011921ms

Example:

product	quantity	price	total_amount
Laptop	2	55000	110000
Mouse	5	800	4000

32. Filter Data

Find transactions from Pune:

<Python>

```
pune_data = df.filter(
    col("city") == "Pune"
)

pune_data.show()
```

#After Run

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 1052ms

33. Group Data

Calculate total sales by city:

<Python>

```
city_sales =
df.groupBy("city").sum("total_amount")

city_sales.show()
```

#After Run

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 1023.3999999761581ms

34. Sort Data

<Python>

```
city_sales.orderBy(
    "sum(total_amount)",
    ascending=False
).show()
```

#After Run

Run started
Initializing environment
Installing packages
Running code
NameError: name 'city_sales' is not defined
Run completed in 1025.5999999642372ms

35. Complete Transformation Program

<Python>

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum

spark = SparkSession.builder \
    .appName("DataTransformation") \
    .getOrCreate()

input_path = "hdfs://localhost:9000/bigdata/assignment1/input/sales.csv"

df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv(input_path)

print("Original Data:")
df.show()
```

```
# Create total amount
df = df.withColumn(
    "total_amount",
    col("quantity") * col("price")
)

print("Data after transformation:")
df.show()

# Total sales by city
city_sales = df.groupBy("city") \
    .agg(sum("total_amount").alias("total_sales"))

print("Sales by city:")
city_sales.show()

# Sort
city_sales.orderBy(
    col("total_sales").desc()
).show()

spark.stop()
```

#After Run
Run started
Initializing environment
Installing packages
Running code
ModuleNotFoundError: No module named 'pyspark'
Run completed in 1563.800000011921ms

PART H — STORE PROCESSED DATA IN MONGODB

36. What is MongoDB?

MongoDB is a NoSQL database.

Instead of storing data in traditional relational tables, MongoDB stores data as documents.

Example:

</>JSON

```
{
    "transaction_id": 1001,
    "product": "Laptop",
    "quantity": 2,
    "price": 55000,
    "city": "Pune",
    "total_amount": 110000
}
```

37. Why Use MongoDB?

MongoDB is useful when:

- Data is semi-structured.
- Schema may change.
- Large amounts of data need to be handled.
- JSON-like documents are convenient.
- Flexible data models are required.

38. MongoDB Structure

MongoDB uses:

```
Database
   |
   +---- Collection
            |
            +---- Document
            +---- Document
            +---- Document
```

For example:

```
Database: BigDataDB

Collection: sales
```

39. Install MongoDB
Install MongoDB Community Edition on the machine or use a MongoDB cloud deployment.
After installation, verify the MongoDB service is running.

40. MongoDB Spark Connector
To write Spark DataFrames directly to MongoDB, the appropriate **MongoDB Spark Connector** must be available to Spark.
The connector version must be compatible with the installed Spark and Scala versions.
This is important because simply installing `pymongo` does **not** provide the Spark DataFrame connector.

41. Write Data to MongoDB

Conceptually, the Spark code is:

<Python>

```
df.write \
    .format("mongodb") \
    .mode("overwrite") \

.option("spark.mongodb.write.connection.uri
",

"mongodb://localhost:27017/BigDataDB.sales"
) \
    .save()
```

Depending on the connector version, the configuration syntax can differ.

#After Run
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df' is not defined
Run completed in 1017.1000000238419ms

42. Complete Example

<Python>

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder \
    .appName("MongoDBIntegration") \
    .getOrCreate()

input_path =
"hdfs://localhost:9000/bigdata/assignment1/
input/sales.csv"

df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv(input_path)
```

```
# Transformation
df = df.withColumn(
    "total_amount",
    col("quantity") * col("price")
)

# Write to MongoDB
df.write \
    .format("mongodb") \
    .mode("overwrite") \
    .option(

"spark.mongodb.write.connection.uri",

"mongodb://localhost:27017/BigDataDB.sales"
    ) \
    .save()

print("Data successfully stored in
MongoDB")

spark.stop()
```

#After Run
Run started
Initializing environment
Installing packages
Running code
ModuleNotFoundError: No module named 'pyspark'
Run completed in 1542.6000000238419ms

PART I — VERIFY DATA IN MONGODB

Open MongoDB Shell or MongoDB Compass.

Select:

`BigDataDB`

Then:

`sales`

You should see documents similar to:

</>JSON

```
{
    "transaction_id": 1001,
    "product": "Laptop",
    "category": "Electronics",
    "quantity": 2,
    "price": 55000,
    "city": "Pune",
    "total_amount": 110000
}
```

43. End-to-End Practical Workflow

The complete workflow is:

```
Step 1
Create/download dataset
        ↓
Step 2
Start Hadoop
        ↓
Step 3
Create HDFS directory
        ↓
Step 4
Upload CSV/JSON to HDFS
        ↓
Step 5
Read data using Spark
        ↓
Step 6
Convert CSV → Parquet
        ↓
```

Step 7
Convert CSV → ORC
↓
Step 8
Compare storage sizes
↓
Step 9
Compare query performance
↓
Step 10
Perform Spark transformations
↓
Step 11
Write processed data to MongoDB
↓
Step 12
Verify MongoDB collection

44. Suggested Final Program

Students can combine the major steps into one program:

<Python>

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum
import time

spark = SparkSession.builder \
    .appName("BigDataAssignment1") \
    .getOrCreate()
```

```
# 1. Read CSV from HDFS
# ----------------------------

input_path =
"hdfs://localhost:9000/bigdata/assignment1/input/sales.csv"

df = spark.read \
   .option("header", True) \
   .option("inferSchema", True) \
   .csv(input_path)

print("Original Dataset:")
df.show()

# ----------------------------
```

```
# 2. Save as Parquet
# ----------------------------

parquet_path =
"hdfs://localhost:9000/bigdata/assignment1/parquet"

df.write \
   .mode("overwrite") \
   .parquet(parquet_path)

print("Parquet file created.")

# ----------------------------
```

```
# 3. Save as ORC
# ----------------------------

orc_path = "hdfs://localhost:9000/bigdata/assignment1/orc"

df.write \
   .mode("overwrite") \
   .orc(orc_path)

print("ORC file created.")

# ----------------------------
```

```
# 4. Transformation
# ----------------------------

df = df.withColumn(
    "total_amount",
    col("quantity") * col("price")
)

print("Transformed Dataset:")
df.show()

# ----------------------------
```

```
# 5. Grouping
# ----------------------------

city_sales = df.groupBy("city") \
    .agg(sum("total_amount").alias("total_sales"))

print("Total Sales by City:")
city_sales.show()

# ----------------------------
```

```
# 6. Save transformed data
# ----------------------------

output_path =
"hdfs://localhost:9000/bigdata/assignment1/processed"

df.write \
   .mode("overwrite") \
   .parquet(output_path)

print("Processed data stored in HDFS.")

# ----------------------------
```

```
# 7. MongoDB
# ----------------------------

df.write \
    .format("mongodb") \
    .mode("overwrite") \
    .option(
        "spark.mongodb.write.connection.uri",
        "mongodb://localhost:27017/BigDataDB.sales"
    ) \
    .save()

print("Data stored in MongoDB.")

spark.stop()
```

45. Expected Output

The output should approximately look like:

```
Original Dataset:

+--------------+--------+-----------+--------+-----+------+
|transaction_id| product|   category|quantity|price|  city|
+--------------+--------+-----------+--------+-----+------+
|          1001|  Laptop|Electronics|       2|55000|  Pune|
|          1002|   Mouse|Electronics|       5|  800|Mumbai|
|          1003|   Chair|  Furniture|       3| 4500| Delhi|
+--------------+--------+-----------+--------+-----+------+
```

```
Parquet file created.

ORC file created.

Transformed Dataset:

+--------------+-------+-----------+--------+-----+------+------------+
|transaction_id| product|   category|quantity|price|  city|total_amount|
+--------------+-------+-----------+--------+-----+------+------------+
|          1001|  Laptop|Electronics|       2|55000|  Pune|      110000|
|          1002|   Mouse|Electronics|       5|  800|Mumbai|        4000|
+--------------+-------+-----------+--------+-----+------+------------+

Data stored in MongoDB.
```

46. Observation Table

Students should fill this table using their actual experimental results.

Parameter	CSV	Parquet	ORC
File size	_____	_____	_____
Read time	_____	_____	_____
Query time	_____	_____	_____
Compression	_____	_____	_____
Storage efficiency	_____	_____	_____

Expected general observation

Parquet and ORC normally require **less storage than uncompressed CSV** because they use efficient columnar encoding and compression. For analytical queries that access only selected columns, Parquet/ORC can also be faster because they can avoid reading unnecessary columns.

47. Result

Result:

The dataset was successfully ingested into HDFS and processed using Apache Spark. The input data was converted into Parquet and ORC formats, and their storage size and query performance were compared. Basic transformations such as filtering, column creation, grouping, and aggregation were performed using PySpark. Finally, the processed data was stored successfully in MongoDB, demonstrating an end-to-end Big Data ingestion, processing, and NoSQL storage pipeline.

48. Important Viva Questions and Answers

Q1. What is Big Data?

Answer:

Big Data refers to extremely large, rapidly generated, and diverse datasets that cannot be efficiently handled using traditional data-processing systems.

Q2. What is Hadoop?

Answer:

Hadoop is an open-source framework used for distributed storage and processing of large datasets across clusters of computers.

Q3. What is HDFS?
Answer:
HDFS stands for Hadoop Distributed File System. It provides distributed storage for large datasets by dividing files into blocks and storing them across DataNodes.

Q4. What is the function of NameNode?
Answer:
The NameNode manages HDFS metadata, including file names, directories, permissions, and the locations of data blocks.

Q5. What is the function of DataNode?
Answer:
DataNodes store the actual blocks of data and serve read/write requests from clients.

Q6. What is data ingestion?
Answer:
Data ingestion is the process of collecting data from different sources and transferring it into a storage or processing system such as HDFS.

Q7. What is Apache Spark?
Answer:
Apache Spark is a distributed data-processing engine designed for fast processing of large datasets.

Q8. What is PySpark?
Answer:
PySpark is the Python API for Apache Spark that allows developers to write Spark applications using Python.

Q9. What is Parquet?
Answer:
Parquet is a column-oriented storage format designed for efficient storage and analytical processing of large datasets.

Q10. What is ORC?
Answer:
ORC stands for Optimized Row Columnar. It is a columnar storage format optimized for efficient storage and querying in Hadoop-based environments.

Q11. Why are Parquet and ORC usually smaller than CSV?
Answer:
They use columnar encoding and compression techniques, which can significantly reduce storage requirements.

Q12. What is MongoDB?
Answer:
MongoDB is a NoSQL document-oriented database that stores data in flexible, JSON-like documents.

Q13. Why is MongoDB called a NoSQL database?
Answer:
MongoDB does not require the traditional fixed table-row-column relational model. It stores records as documents in collections.

Q14. What is a DataFrame in Spark?
Answer:
A DataFrame is a distributed collection of data organized into named columns, similar to a table in a relational database.

Q15. What is `groupBy()` used for?

Answer:

`groupBy()` is used to group records according to one or more columns so that aggregate operations such as sum, count, average, minimum, or maximum can be performed.

Example:

<Python>

```
df.groupBy("city").sum("total_amount")
```

#After Run

Run started
Initializing environment
Installing packages
Running code
NameError: name 'df' is not defined
Run completed in 1032.300000011921ms

Q16. What is the difference between CSV and Parquet?

CSV	Parquet
Row-oriented/text format	Column-oriented/binary format
Usually larger	Usually smaller
Human-readable	Not directly human-readable
Less efficient for analytics	Efficient for analytics
Limited type information	Preserves data types

Q17. What is the difference between HDFS and MongoDB?

HDFS	MongoDB
Distributed file system	NoSQL database
Primarily storage for large files	Document-oriented storage
Hadoop ecosystem	NoSQL ecosystem
Files/blocks	Documents/collections
Suitable for large-scale batch processing	Flexible application/database workloads

49. Practical Examination Questions

Students should be able to perform the following independently:

Task 1

Create an HDFS directory.

<Bash>

```
hdfs dfs -mkdir -p /bigdata/assignment1/input
```

Task 2
Upload a CSV file.
<Bash>

```
hdfs dfs -put sales.csv /bigdata/assignment1/input/
```

Task 3

Display the file.

<Bash>

```
hdfs dfs -cat /bigdata/assignment1/input/sales.csv
```

Task 4

Read it using PySpark.

<Python>

```
df = spark.read.option(
    "header", True
).option(
    "inferSchema", True
).csv(input_path)
```

#After Run

Run started

Initializing environment

Installing packages

Running code

NameError: name 'spark' is not defined

Run completed in 1055.9000000357628ms

Task 5

Convert it to Parquet.

<Python>

```
df.write.parquet("parquet_output")
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 977.5ms

Task 6

Convert it to ORC.

<Python>

```
df.write.orc("orc_output")
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 1033.3999999761581ms

Task 7

Create a calculated column.

<Python>

```
df = df.withColumn(
    "total",
    col("quantity") * col("price")
)
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 999.1000000238419ms

Task 8

Perform aggregation.

<Python>

```
df.groupBy("city").sum("total")
```

#After Run

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 1060.800000011921ms

Task 9

Store the result in MongoDB.

<Python>

```
df.write.format("mongodb").save()
```

#After Run

Run started

Initializing environment

Installing packages

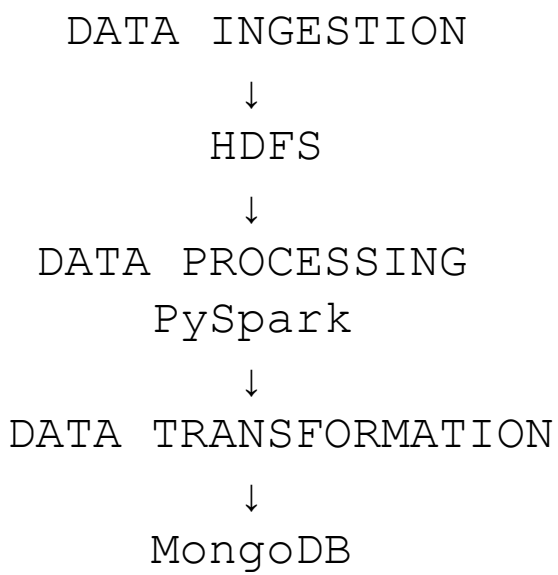
Running code

NameError: name 'df' is not defined

Run completed in 994.4000000357628ms

50. Important Note for Students

There are **four different stages** in this assignment:



Do not confuse their purposes:

- **HDFS** → stores large files.
- **Spark** → processes/transforms data.
- **Parquet/ORC** → efficiently store analytical data.
- **MongoDB** → stores processed data as NoSQL documents.

This distinction is one of the **most important concepts to understand for the practical examination**.